# Analyse v1 → v16 — Trajectoire complète de l'optimisation Phase C

**Note du 11/11/2026** — le chiffre « 96 manquants » est celui du journal de juin 2026. Les **97 manquants** du jeu V13 à T=$5 \cdot 10^6$ ont été retrouvés et fusionnés le 05/11/2026 (10 016 474 zéros, Turing complet) ; cause mesurée : 84 vraies paires supprimées par `dedupliquer(0,01)` + 13 zéros isolés (voir [[JOURNAL]] 2026-11-05 et [[analyse_problemes_v16_v17]]). Le texte ci-dessous est conservé tel quel (historique daté).

**Note du 11/11/2026 (gains cumulés)** — les gains « depuis v1 » de ce document comparent 21 h (mesurées à T=10 000) à des durées à T=100 000 : ils ne sont pas comparables, et certains (×4 500, ×5 040, ×5 628, ×28 600+) ne sont pas reproductibles (21 h divisées par la durée donnent ×53 à ×286). Référence retenue : comparaison à T égal, v2 → v16 ≈ ×7 490 à T=10 000 — voir [[Etape-1-Calcul-des-zéros-non-triviaux]], « Gains cumulés à T égal ». Le « ≈ ×78 650 » du résumé correspond à 21 h / 1,6 min = ×787,5 en valeur brute (facteur 100 erroné). Le texte ci-dessous est conservé tel quel (historique daté).

**Date :** 2026-08-08
**Branche :** Riemann_Lab_C (synchronisée main / Riemann_Lab_IA / Riemann_Lab_Test)
**Auteur :** hprzeta

---

## 0. Résumé exécutif

Ce rapport synthétise la trajectoire complète d'optimisation du calcul des zéros non-triviaux de $\zeta(s)$, de la version de référence naïve (v1, ~21h pour 10 000 zéros) à la version de production actuelle (v16, 1.6 min pour 138 069 zéros à T=100 000). Chaque transition majeure fait l'objet d'un rapport dédié (`analyse_problemes_vX_vY.md`, listés en fin de document) ; ce document resitue l'ensemble dans une seule trajectoire et développe en détail la transition la plus récente (v15 → v16), conformément à la structure des rapports précédents.

**Gain global mesuré : v1 → v16 ≈ ×78 650** (21h → 1.6 min, T=100 000 zéros). **Condition Objectif 2 (T=100k < 5 min) : atteinte le 04/07/2026 (v15, 4.4 min), largement dépassée le 08/08/2026 (v16, 1.6 min).**

| Étape | Version | Levier principal | Gain vs étape précédente |
|---|---|---|---|
| 1 | v1 → v3 | Illinois + parallélisation (W=4) | ×28 (T=10k) |
| 2 | v3 → v4.1 | Illinois C/libmpfr (ctypes) | ×5 (T=10k) |

| Étape | Version | Levier principal | Gain vs étape précédente |
|---|---|---|---|
| 3 | v4.1 → v5/v6 | Arb (`arb_fpwrap_cdouble_hardy_z`), STEP adaptatif | ×9 (T=10k) |
| 4 | v6 → v7 | `prec_fast=64 bits` → 1 limbe → SIMD | ×4.2 (T=100k) |
| 5 | v7 → v10 | Brent IQI + W=8 forcé | ×1.7 cumulé (T=100k) |
| 6 | v10 → v12 | Illinois hybride 2-phases (Z_rs + Newton Arb) | ×2.69 (T=100k) |
| 7 | v12 → v14 | T_SEUIL 200→65, TOL 1e-12, cache RS statique | ×1.14 cumulé (T=100k) |
| 8 | v14 → v15 | Phase 2 adaptative (SEUIL_1NEWTON=20k) | ×1.93 vs v13 (T=100k) |
| **9** | **v15 → v16** | **Z_arb à précision fixe (`acb_dirichlet_hardy_z`)** | **×2.75 (T=100k)** |

---

# 1. v1 → v11 — Des fondations à Arb (résumé)

## 1.1 v1 → v3 — Méthode et parallélisation

v1 : Newton scalaire séquentiel, 1 worker, ~21h pour 10 000 zéros. v2 introduit la méthode d'Illinois (sécante corrigée, convergence super-linéaire ~1.44 vs 1.0 pour la bissection) et la fonction de Hardy $Z(t) = e^{i\theta(t)}\zeta(\frac{1}{2}+it)$. v3 parallélise sur W=4 workers (découpage T/W, chargement post-fork du `.so`, correction de la formule N(T) — le terme `e` dans $2\pi e$ est obligatoire, son omission sous-estime N(T) de 64 % à T=100 000). **Gain : 21h → 45 min (×28).**

## 1.2 v3 → v4.1 — Illinois en C/libmpfr

Portage de l'affinage Illinois en C (`illinois_mpfr.c`, PREC=170 bits), appelé depuis Python via `ctypes` avec `(f_a, f_b)` pré-calculés côté Python (Option B — évite un recalcul incohérent des bornes en C). **Gain : 45 min → 9 min (×5).**

## 1.3 v4.1 → v5/v6 — Arb et STEP adaptatif

Remplacement de l'affinage MPFR manuel par `arb_fpwrap_cdouble_hardy_z` (FLINT/Arb, double natif IEEE 754, 0 malloc) — gain ×27 mesuré vs mpmath en benchmark isolé. STEP de détection rendu adaptatif (0.05 pour t<5k, 0.010 au-delà, overlap fixe) pour éviter les zéros manqués sur grille trop grossière à grand t. **Gain : 9 min → 2.60 min à T=10 000 (×3.5), Turing-Backlund COMPLET.**

## 1.4 v6 → v7 — SIMD via précision réduite

Découverte contre-intuitive (leçon durable, cf. `Bonnes-Pratiques-Claude-Code.md` § Règle benchmark avant implémentation) : réduire le nombre de termes RS ($N_{\text{fast}} = N_{\text{full}}/4$) invalide les signes

de $Z(t)$ pour $N_{\text{full}} > 10$ — hypothèse théorique fausse. Le vrai levier, trouvé par benchmark systématique plutôt que par intuition : `prec_fast = 64 bits` → tient sur 1 seul limbe MPFR → auto-vectorisation SIMD AVX2 du compilateur. **Gain ×16 en benchmark isolé de la Phase 1, ×4.2 global (T=100k : 130 min → 30.9 min).**

### 1.5 v7 → v10 — Brent et parallélisme matériel

v8 teste `prec_full` 116→80 bits (gain marginal ×1.06, plancher hardware atteint sur l'i7-7500U du matériel de l'époque). v9 remplace la sécante-Illinois par une méthode de Brent (IQI + sécante + bissection, ordre de convergence ~1.84) en C/MPFR — gain ×1.80 vs v8. v10 force W=8 workers (au lieu de W=4, malgré seulement 4 cœurs physiques / 8 logiques) : le coût élevé des évaluations Brent à grand t (~64 ms/appel) dilue l'overhead de context-switch de l'hyperthreading — gain ×1.12. **Cumulé v7→v10 : 30.9 min → 23.7 min (T=100k, avec turbo).**

*(Détail complet de ces 5 transitions : `analyse_problemes_v2_v3_phase0.md` à `analyse_problemes_v9_v10.md`, un rapport par transition.)*

---

## 2. v10 → v12 — Illinois hybride 2-phases (résumé)

**Rupture architecturale :** au lieu d'affiner directement avec Brent/MPFR (coûteux à chaque itération), v12 introduit une architecture à **2 phases** qui reste la base de toutes les versions suivantes (v12 à v16) :

- **Phase 1 — Illinois avec `Z_rs_double`** (Riemann-Siegel tronquée en double, quasi gratuite, ~0.015 ms/appel) : converge vers un *pseudo-zéro* biaisé jusqu'à un bracket serré ($|b - a| < 10^{-6}$).
- **Phase 2 — Newton avec `Z_arb`** (Arb/FLINT, coûteux mais précis) : 1 à 2 pas de Newton depuis le pseudo-zéro Phase 1 pour converger vers le vrai zéro à tol $= 10^{-12}$.

**Résultat :** l'essentiel du travail (convergence grossière) se fait sur la fonction quasi-gratuite ; la fonction coûteuse n'est appelée que 2 à 4 fois par zéro au lieu de dizaines d'itérations Brent complètes. **Gain ×2.69 vs v10, ×16.9 en benchmark isolé de l'affinage seul (T=100k : 23.7 min → 8.8 min).**

*(Détail complet : `analyse_problemes_v10_v12.md`.)*

---

## 3. v12 → v14 — Seuils et cache RS (résumé)

v13 abaisse `T_SEUIL_PETIT_T` de 200 à 65 (le RS a des signes fiables dès $N_{\text{RS}} = 3$ termes, soit $t \geq 65$) et resserre `TOL_ARB` de $10^{-9}$ à $10^{-12}$ (le seuil précédent était trop proche du critère de validation LMFDB et causait des faux 16/20). v14 précompute `log(n)` et `1/\sqrt n\)` dans des tableaux statiques (`log_n_cache`, `isqrt_n_cache`, 33 KB, tient en cache L2, initialisés une fois par worker post-fork) — remplace ~100 cycles/terme (`log()`+`sqrt()`) par ~4 cycles (lecture L2). **Cumulé v12→v14 : 8.8 min → 7.7 min (×1.14, T=100k).**

*(Détail complet : `analyse_problemes_v13_v15.md` §1-2.)*

---

## 4. v14 → v15 — Phase 2 adaptative SEUIL_1NEWTON

La Phase 2 (v12-v14) exécute systématiquement 2 pas de Newton, quel que soit $t$. v15 établit que le biais de `Z_rs_double` suit biais$(t) \approx C \cdot t^{-5/4}$ ($C \approx 0.305$, calibré sur LMFDB), et que l'erreur après **1 seul** pas de Newton ($\varepsilon \approx C^2 \cdot t^{-5/2}$) descend sous la tolérance $10^{-12}$ dès $t \gtrsim 16\,000$ :

| $t$ | biais$(t)$ | $\varepsilon$ 1 Newton | $<$ tol ? |
|---|---|---|---|
| 65 | $5.0 \times 10^{-3}$ | $1.75 \times 10^{-6}$ | × |
| 10 000 | $3.7 \times 10^{-6}$ | $9.8 \times 10^{-12}$ | (!) limite |
| **20 000** | $6.4 \times 10^{-7}$ | $4 \times 10^{-13}$ | ✓ (marge ×2.4) |
| 100 000 | $9.6 \times 10^{-9}$ | $9.2 \times 10^{-17}$ | ✓ |

`SEUIL_1NEWTON = 20 000` retenu (87 % des zéros de T=100k dépassent ce seuil, économisant chacun 1 appel `Z_arb`). **Piège confirmé et documenté :** réduire à 1 Newton pour *tout* $t$ donne LMFDB 14/20 (erreur $1.75 \times 10^{-6}$ à $t \approx 65$) — la règle adaptative est impérative, pas une optimisation optionnelle. **Gain v15 vs v13 : ×1.93 (T=100k : 8.50 min → 4.4 min). Condition Objectif 2 (T=100k < 5 min) atteinte le 04/07/2026.**

*(Détail complet, dérivation mathématique complète : `analyse_problemes_v13_v15.md` §3, `Formules_zeta.md` §30.)*

---

## 5. v15 → v16 — Z_arb à précision fixe (`acb_dirichlet_hardy_z`)

### 5.1 Diagnostic — le vrai coût de `arb_fpwrap_cdouble_hardy_z`

Après v15, la Phase 2 (`Z_arb`, appelée via `arb_fpwrap_cdouble_hardy_z(flags=0)`) reste le goulot dominant. Plutôt que de deviner un nouveau levier, un profiling réel a été mené : `perf record` sur 300 appels représentatifs de `illinois_refine_arb` (2 zones — $t \sim [500, 5000]$ branche 2-Newton, $t \sim [50000, 150000]$ branche 1-Newton — 85 232 échantillons à 4000 Hz).

**Résultat du profiling :**

| Bibliothèque | Part du temps (self) |
|---|---|
| libgmp (arithmétique bignum) | 37.50 % |
| libflint (Arb, ball arithmetic) | 53.53 % |
| **Total GMP+FLINT (via `arb_fpwrap`)** | **91 %** |
| Notre code (`illinois_arb.so`, Phase 1) | 0.20 % |

Fonction individuelle la plus chaude : `__gmpn_mul_basecase_coreisbr` (multiplication bignum brute) — **16.67 % à elle seule**. Le mécanisme interne a été confirmé en lisant le code source de FLINT 3.3.1 (`src/arb_fpwrap/fpwrap.c`, version runtime identifiée via le symbole `flint_version`) :

```
for (wp = 64; ; wp *= 2) {          /* 64 -> 128 -> ... -> 8192 bits (flags=0) */
    func(arb_res, arb_x, wp);       /* recalcul COMPLET à chaque niveau */
    if (arb_accurate_enough_d(arb_res, flags)) return FPWRAP_SUCCESS;
    if (wp >= double_wp_max(flags)) return FPWRAP_UNABLE;
}
```

`arb_fpwrap` vise systématiquement une précision certifiée ~$10^{-16}$ (un `double` complet) — largement plus que le besoin réel du pipeline (`TOL_ARB = 10^{-12}`, soit ~40 bits utiles, pas 53). **Piste**

**`flags` écartée** : `flags=0` (déjà utilisé) est déjà la variante la moins coûteuse de l'API `fpwrap` — `FPWRAP_ACCURATE_PARTS`/`FPWRAP_CORRECT_ROUNDING` ne font qu'augmenter le coût, aucun réglage de `flags` ne permet de viser une précision plus basse.

### 5.2 Piste écartée — MPFR pur

Avant de converger vers la solution retenue, une comparaison directe a été menée contre `illinois_refine` (MPFR pur, PREC=170 bits fixe, sans Arb — réévalue `Z_mpfr` à chaque itération de la sécante au lieu de 1-2 fois) sur les mêmes 300 brackets :

| Méthode | ms/appel | Erreur moyenne vs `mpmath` dps=35 |
|---|---|---|
| `illinois_refine_arb` (Arb) | 2.95 | $4.6 \times 10^{-11}$ |
| `illinois_refine` (MPFR pur) | 35.05 | $9.5 \times 10^{-5}$ |

**MPFR pur est 11.88× plus lent ET ~4 ordres de grandeur moins précis** — le biais vient de la formule (Riemann-Siegel tronquée $C_0$+$C_1$), pas de la précision de calcul ; monter la précision MPFR n'élimine pas ce biais structurel. Cette piste est **écartée définitivement**.

### 5.3 Solution retenue — `acb_dirichlet_hardy_z` à précision fixe

FLINT expose une fonction bas niveau avec précision **fixe explicite, un seul calcul, sans boucle d'escalade** :

```
void acb_dirichlet_hardy_z(acb_ptr res, const acb_t t,
    const dirichlet_group_t G, const dirichlet_char_t chi,
    slong len, slong prec);
```

Contrairement à `arb_fpwrap`, cette fonction nécessite `dirichlet_group_t`/ `dirichlet_char_t` — même pour la fonction zêta pure, il faut initialiser le caractère principal du groupe trivial modulo 1 ($q = 1$), une fois par worker (pattern confirmé sur le test officiel FLINT `acb_dirichlet/test/t-hardy_z.c`) :

```
static void init_dirichlet_trivial(void) {
    if (g_dirichlet_ready) return;
    dirichlet_group_init(g_G, 1);
    dirichlet_char_init(g_chi, g_G);
    dirichlet_char_index(g_chi, g_G, 0);   /* caractère principal mod 1 */
    g_dirichlet_ready = 1;
}

#define PREC_BITS_ARB 64

static double Z_arb(double t) {
    init_dirichlet_trivial();
    acb_t s, res;
    acb_init(s); acb_init(res);
    acb_set_d(s, t);
    acb_dirichlet_hardy_z(res, s, g_G, g_chi, 1, (slong)PREC_BITS_ARB);
    double result = arf_get_d(arb_midref(acb_realref(res)), ARF_RND_NEAR);
    acb_clear(s); acb_clear(res);
    return result;
}
```

### 5.4 Nouvelle dépendance — headers FLINT vendorisés en source

`acb_dirichlet_hardy_z` nécessite les vrais types internes FLINT, pas seulement une déclaration `extern` comme pour `arb_fpwrap`. `apt install libflint-dev` ne fournit que la version **3.0.1**, alors que la `.so`

réellement chargée en runtime (bundlée par le paquet Python `python-flint`) est **3.3.1** — vérifié via le symbole `flint_version`. `dirichlet_group_t`/`dirichlet_char_t` étant des types activement modifiés entre versions FLINT, compiler avec des headers 3.0.1 contre une `.so` 3.3.1 aurait risqué une **corruption mémoire silencieuse** (mauvais offsets de champs de structure), pas juste un mauvais résultat.

**Solution :** headers récupérés depuis les sources officielles (tag `v3.3.1`), `./configure` local pour générer `flint.h`/`flint-config.h` corrects, extraction de la seule fermeture transitive réellement utilisée (`gcc -M`) — **23 headers, 276 Ko** au lieu des 800+ de FLINT complet. Vendorisés dans `c_modules/flint-headers-3.3.1/` (provenance et licence LGPL documentées), utilisés uniquement à la compilation — l'édition de liens se fait contre la `.so` déjà présente dans l'environnement Python.

### 5.5 Choix de la précision

| Précision | Gain vs `arb_fpwrap` (300 brackets) | Écart vs `arb_fpwrap` |
|---|---|---|
| 48 bits | ×8.82 | 0 — identique bit-à-bit |
| **64 bits** | **×5.75** | **0 — identique bit-à-bit** |
| 128 bits | ×1.21 | identique |

**64 bits retenu** plutôt que 48 (gain max) : coïncide avec `WP_INITIAL` de `arb_fpwrap` lui-même, marge ~7 décimales au-dessus du besoin théorique ~40 bits (tol=$10^{-12}$) contre ~2.4 décimales à 48 bits — jugé plus sûr face à un cas limite non couvert par l'échantillon de 300 brackets testé.

### 5.6 Validation en 2 étapes avant intégration production

Protocole appliqué (voir `Bonnes-Pratiques-Claude-Code.md` § Workflow de validation avant intégration production) : isolation stricte du code de production pendant le prototypage, validation sur run réel avant toute décision d'intégration, puis revalidation au protocole standard (T=100k) avant d'adopter la version.

**Étape 1 — Prototype isolé, run réel T=10000** (`illinois_arb.c` de production non touché) :

| | Production (`arb_fpwrap`) | Prototype (64 bits fixe) |
|---|---|---|
| Zéros | 10 142 | 10 142 (identiques) |
| Turing-Backlund | COMPLET | COMPLET |
| LMFDB | 20/20 | 20/20 |
| Durée | 20.2 s | **10.2 s** |
| **Gain** | — | **×1.98** |

**Étape 2 — Intégration, revalidation T=100000** (protocole standard Objectif 2, identique à celui appliqué pour v12→v15) :

| | v15 (référence) | **v16 (intégré)** |
|---|---|---|
| Zéros | 138 069 | **138 069 / 138 069** (exact, Weyl) |
| Durée | 4.4 min | **1.6 min (98.1 s)** |
| Vitesse | 517 z/s | **1407 z/s** |
| Turing-Backlund | COMPLET | **COMPLET** |
| LMFDB | 20/20 | **20/20** |

| | v15 (référence) | **v16 (intégré)** |
|---|---|---|
| **Gain** | — | **×2.75** |

L'écart entre ×1.98 (T=10000) et ×2.75 (T=100000) s'explique par la part relative croissante de la phase Illinois dans le temps mur total à mesure que $T$ augmente (profil détaillé du run T=100000 : `arb_C` 334.81 s cumulés / 138 061 appels = 2.43 ms/appel, contre 12.28 ms/appel en v15) — cohérent avec la mesure initiale de 91 % du temps dans GMP+FLINT.

### 5.7 Découverte annexe (sans impact production)

En validant contre `test_illinois_arb.py` (20 premiers zéros LMFDB, appel isolé hors pipeline), 3 échecs constatés à $t < 300$ (17/20 au lieu de 20/20 documenté) — reproduits à l'identique par le prototype, donc préexistants, pas une régression. Confirmé sans impact sur le pipeline réel : `compute_zeros_v16.py` route déjà $t < 65$ via un chemin séparé (`arb_hardy_z` + `mpmath`), jamais via `illinois_refine_arb` directement — vérifié LMFDB 20/20 sur les runs T=1000 et T=100000 réels.

---

## 6. Tableau récapitulatif global — T = 100 000 zéros

| Version | Temps | Vitesse | Gain vs précédente | Zéros | Turing | LMFDB | Commit |
|---|---|---|---|---|---|---|---|
| v1 (STEP=0.1 fixe) | ~1h58 | ~19.5 z/s | baseline | 137 904 | — | — | — |
| v2 (STEP adaptatif) | ~105 min | ~22 z/s | — | 138 039 | × INCOMPLET | — | — |
| v3 (STEP 0.05/0.010) | ~113 min | ~20 z/s | — | 138 069 | ✓ COMPLET | — | `181fdd1` |
| v6 (Z_double C inline) | ~130 min | ~17.7 z/s | — | 138 069 | ✓ COMPLET | — | `b676e88` |
| v7 (prec=64b SIMD) | 30.9 min | 74.5 z/s | ×4.2 vs v6 | 138 069 | ✓ COMPLET | — | `8637098` |
| v8 (prec_full=80b) | 50.5 min* | 45.6 z/s | ×1.06 vs v7 | 138 069 | ✓ COMPLET | — | — |
| v9 (Brent C/mpfr) | 26.6 min | 86.5 z/s | ×1.80 vs v8 | 138 069 | ✓ COMPLET | — | `1c8c0ab` |
| v10 (W=8 forcé) | 23.7 min | 97.1 z/s | ×1.12 vs v9 | 138 069 | ✓ COMPLET | — | `c9fad76` |
| v12 (Illinois hybride) | 8.8 min | 261 z/s | ×2.69 vs v10 | ~138 080 | ✓ COMPLET | 20/20 | `f0e8430` |

| Version | Temps | Vitesse | Gain vs précédente | Zéros | Turing | LMFDB | Commit |
|---|---|---|---|---|---|---|---|
| v13 (T_SEUIL=65) | 8.50 min | 271 z/s | ×1.04 vs v12 | 138 069 | ✓ COMPLET | 20/20 | `77efd10` |
| v14 (cache RS) | 7.7 min | 299 z/s | ×1.10 vs v13 | 138 069 | ✓ COMPLET | 20/20 | `d4b3611` |
| v15 (SEUIL_1NEWTON=20k) | 4.4 min | 517 z/s | ×1.93 vs v13 | 138 069 | ✓ COMPLET | 20/20 | `adf5d2a` |
| **v16 (Z_arb précision fixe)** | **1.6 min** | **1407 z/s** | **×2.75 vs v15** | **138 069** | **✓ COMPLET** | **20/20** | `00abe5c` |

*v9 : gain turbo ×1.05 seulement — bottleneck MPFR/mémoire (vs ×1.63 pour v7).*

**Gains globaux cumulés depuis v1 (T=10 000, baseline 21h Newton scalaire) :**

| Jalon | T=100k | Gain v1→jalon |
|---|---|---|
| v9 (sans turbo) | 28 min | ×4 300 |
| v9 (turbo) | 26.6 min | ×4 500 |
| v10 (turbo) | 23.7 min | ×5 040 |
| v13 distribué PC1+PC2 | 8.5 min mur | ×20 000+ |
| v15 (PC1 local) | 4.4 min | ×28 600+ |
| **v16 (PC1 local)** | **1.6 min** | **×78 650+** |

---

## 7. Questions ouvertes

- **Run T=5M avec v16 :** le gain ×2.75 mesuré à T=100k n'a pas encore été mesuré à l'échelle T=5M (run v15 arrêté le 06/08/2026, PC2 goulot d'étranglement — décision de reprise toujours en attente). Le gain à T=5M pourrait différer : le profil des appels `Z_arb` (1 vs 2 Newton selon `SEUIL_1NEWTON`) et la part relative de la phase Illinois évoluent avec $T$.
- **Investigation des 96 manquants (v13, run T=5M du 27-29/06/2026) :** cause identifiée (paires de zéros proches sous la résolution du STEP de scan), pas encore corrigée par un nouveau run. Économie de temps désormais disponible (v16) pour explorer un STEP plus fin ou un rescan ciblé plus large.
- **v17 — Odlyzko-Schönhage** (long terme, non commencée) : $O(N\log^2 N)$ vs $O(N\sqrt{T})$ actuel — gain ×50+ estimé à T=5M, nécessaire pour $T > 10$M. Implémentation complexe (FFT non-uniforme).
- **Cache RS au-delà de T=27M :** `N_MAX_CACHE=2100` actuel ; pour T=50M, $N_{\text{RS}} \approx 2823 \rightarrow$ fallback sans cache. Option : augmenter `N_MAX_CACHE` (48 KB à 3000, toujours en L2) ou `realloc` dynamique.

- **Maintenance de la dépendance FLINT headers :** si `python-flint` est mis à jour dans `zeta_env` et bundle une version FLINT différente de 3.3.1, les headers vendorisés (`c_modules/flint-headers-3.3.1/`) devront être régénérés — sinon risque de décalage ABI silencieux (voir README du dossier).
- **Objectif 2 — agent IA autonome :** condition numérique (T=100k $<$ 5 min) largement dépassée (1.6 min). Prochain jalon : Anthropic Skilljar + MCP + RAG vault (`/mnt/vault_rag`, déjà en service — 838 chunks ingérés).

---

## Rapports détaillés par transition

| Transition | Fichier |
|---|---|
| v1 → v3 | `analyse_problemes_v2_v3_phase0.md` |
| v3 → v4 | `analyse_problemes_v3_v4.md` |
| v4 → v4.1 | `analyse_problemes_v4_v4_1.md` |
| v4.1 → v5 | `analyse_problemes_v4_1_v5.md` |
| v5 → v6 | `analyse_problemes_v5_v6.md` |
| v4.1 → v6 (synthèse) | `analyse_problemes_v4_1_v6_synthese.md` |
| v6 → v7 | `analyse_problemes_v6_v7.md` |
| v7 → v8 | `analyse_problemes_v7_v8.md` |
| v8 → v9 | `analyse_problemes_v8_v9.md` |
| v9 → v10 | `analyse_problemes_v9_v10.md` |
| v10 → v12 | `analyse_problemes_v10_v12.md` |
| v13 → v15 | `analyse_problemes_v13_v15.md` |
| **v15 → v16** | **ce document, §5** |

---

*analyse_problemes_v1_v16.md · Riemann_Lab.wiki/master · hprzeta · MAJ 2026-08-08 · note du 11/11/2026 · 416 lignes*